

[RI/2025/Q1]

1 Name your Jupyter Notebook as:

TASK1_<your name>_<index number>.ipynb

Your school is organising an Inter-House Sports Carnival, where students from five different houses compete in friendly games.

Student participants information is stored in `student_codes.txt`. Each student is identified by a code that combines their house name and student number, using the format:<HouseCode><StudentNumber>

where

- <HouseCode> is one of the 5 houses, in 2 letter code:
 - BW representing Bailey-Waddle
 - BB representing Buckle-Buckley
 - HH representing Hadley-Hullet
 - MR representing Morrision-Richardson
 - MT representing Moor-Tarbet
- <StudentNumber> is an integer between 1 and 99

Examples of student codes: "BW12", "MR5", "HH88"

The organising committee has decided not to group students by house. Instead, the games are organised into two teams based on student numbers:

- Students with even student numbers will join Team Blue
- Students with odd student numbers will join Team Red

For each of the sub-tasks, add a comment statement at the beginning of the code using the hash symbol '#' to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 1.1
        Program code
```

Output:

Your task is to write a program that processes a list of student participants, sorts them into their respective teams using **Insertion Sort**, and generates matchups for parallel rounds of competition.

[RI/2025/Q1 Cont ']

Task 1.1

Write a Python function, `validate_student_code(code_list)`, that takes in a list of student codes and validates them according to these rules:

- The code must begin with a valid house code (BW, BB, HH, MR or MT)
- It must be followed by a number between 1 and 99
- Any invalid codes should be rejected with an appropriate message printed on the console.
- Ensures no duplicates

Examples of invalid codes: MR-5, HH, MT0, mr99

The function should return a list of valid codes. Test your program with appropriate test data.

[4]

Task 1.2

Write a function, `insertion_sort_by_number(code_list)`, to sort a list of student codes using your own Insertion Sort algorithm.

Sort the list by the numeric part of the code, e.g. "MR5" is sorted as 5.

If two student codes have the same number, it breaks the tie by alphabetically sorting the house code. For example, MT5 and HH5, the sorted order should be HH5 followed by MT5.

Do not use built-in sort functions like `sorted()` or `.sort()`

This function will be reused in Task 1.3.

[4]

Task 1.3

Write code to process `student_codes.txt`, and return two lists containing sorted codes of the Team Blue and Team Red.

Use Task 1.1 and 1.2 appropriately to answer this question.

[4]

[RI/2025/Q1 Cont ']

Task 1.4

The games will proceed in parallel rounds to encourage maximum interaction between students from different houses.

Each round is formed as follows:

- The first student from Team Blue is matched with the last student from Team Red
- The second from Team Blue is matched with the second-last from Team Red, and so on

If one team has more students than the other, the unmatched students are paired with "BYE".

For instance,

```
Team Blue = ["BB8", "BW12", "HH14"]
```

```
Team Red = ["HH3", "MR5"]
```

The matchups would be as follows:

```
Round 1: BB8 vs MR5
```

```
Round 2: BW12 vs HH3
```

```
Round 3: HH14 vs BYE
```

Write a function `generate_reverse_matchups(team_blue, team_red)` to produce this output. Run your function.

[5]